SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-033: Last Login Tracking and Display for Student Accounts

Change Request ID	CR-033
Change Request Title	Last Login Tracking and Display for Student Accounts
Student Name	Y.Ugatharsini
Student ID	JF/ICT/24/33
System Name	Student Management System
Implementation Status	Implemented; verified in development according to the user
Report Date	13 October 2026

.

1. Change Request Summary

1.1 Business Requirement

The Student Management System needed to record the most recent successful login time for student accounts and display it in the administrator's student list. This helps an administrator see whether a student account has logged in and when the most recent successful login occurred.

1.2 Existing Behaviour

- Students could authenticate using their existing username and password.
- The Admin Dashboard displayed student details and provided existing student-management actions.
- The system did not previously provide a visible last-login value in the student list.

1.3 Requested Change

- Store a last-login timestamp for a user account after successful authentication.
- Do not update the timestamp when authentication fails.
- Return the timestamp with student details for the dashboard.
- Display a readable date and time in a Last Login column.
- Display “Never logged in” when the timestamp is NULL or unavailable.
- Preserve existing login, token generation, student listing, search, and CRUD functionality.

2. Implementation

2.1 Implementation Summary

The change adds a nullable last_login field to the users table, updates the user model to write the current database timestamp, and calls that update after password verification succeeds. The student model retrieves users. last_login by joining users to students on user_id. The Admin Dashboard presents the value in a dedicated column and displays “Never logged in” when there is no recorded timestamp. The dashboard stylesheet includes styling for the Last Login column and its logged-in / never-logged-in labels.

2.2 Affected Files

File	Change / Purpose
database/schema.sql	Update the users table definition for new installations by adding last_login.
backend/models/userModel.js	Select last_login with user details and implement updateLastLogin(userId).
backend/controllers/authController.js	Call updateLastLogin(user.user_id) only after the password matches.
backend/models/studentModel.js	Include users.last_login in student queries using a join on user_id.
frontend/src/pages/DashboardPage.jsx	Show the Last Login table heading and formatted timestamp / fallback.
Frontend dashboard stylesheet (dashboard.css; exact folder depends on project)	Style the Last Login column and the last-login / never-login labels.

Existing database migration (run once only if the column does not already exist):

```
ALTER TABLE users
ADD COLUMN last_login DATETIME NULL DEFAULT NULL;
```

2.3 Technical Changes

2.3.1 Database

The users table uses last_login DATETIME NULL DEFAULT NULL. DATETIME stores the date and time; NULL represents an account for which no successful login timestamp has been recorded. The ALTER TABLE statement is needed only for an existing database where the column is absent. The schema.sql definition should also be updated for new installations.

2.3.2 User Model

findUserByUsername() selects user_id, username, password_hash, role, and last_login. findUserById() returns the user's identity, role, and last_login. updateLastLogin() executes a parameterized UPDATE statement and returns the database result. The query parameter is passed separately from the SQL.

```
UPDATE users SET last_login = NOW() WHERE user_id = ?;
```

2.3.3 Authentication Controller

The login controller first checks the submitted username and password, retrieves the user, and verifies the password against password_hash using bcrypt. It then calls updateLastLogin(user.user_id) after successful password verification and before creating the JWT. A missing username or incorrect password returns an error before the timestamp update. In the supplied implementation, this update applies to any account that can use this login function; if the requirement is student-only tracking, add a role condition such as if (user.role === 'STUDENT') around the update, using the exact role value configured in the database.

2.3.4 Student Model

getAllStudents(), getStudentById(), and getStudentByUserId() select users.last_login through a LEFT JOIN between students.user_id and users.user_id. A LEFT JOIN allows a student record to remain in the result even if its linked user row is missing. Existing create, update, and delete functions are retained.

2.3.5 Admin Dashboard and CSS

The dashboard table includes a Last Login heading and renders the timestamp for each student. A valid value is formatted for display; a missing value is shown as “Never logged in”. The dashboard stylesheet styles the column and the .last-login and .never-login labels. The nth-child(5) CSS selector is appropriate only while Last Login remains the fifth table column; update the selector if the column order changes.

2.4 Implementation Flow

- The student submits a username and password.
- The backend finds the user and verifies the password hash.
- If authentication fails, the login request returns an error and does not update last_login.
- After successful verification, updateLastLogin(user.user_id) stores NOW() in the database.
- The existing JWT is generated and the successful login response is returned.
- The student-list query retrieves the timestamp with student details.
- The dashboard formats the timestamp or displays “Never logged in” when no timestamp exists.

3. Testing and Verification

3.1 Testing Approach

Manual verification should cover the database field, successful and failed login attempts, the student-list API response, dashboard display, and regression of existing student-management actions. The user indicated that the feature and CSS were working in the development environment; the detailed test cases below should still be checked against actual results before formal submission.

3.2 Test Cases

Test ID	Test Case	Expected Result	Observed / Evidence	Status
TC-01	Check the users table for last_login.	Column exists and permits NULL.	Verify in MySQL	To confirm
TC-02	Log in successfully with a student account.	Timestamp updates to the latest login time.	Reported working; retain DB evidence	Verify evidence
TC-03	Attempt login with an incorrect password.	Timestamp remains unchanged.	Not specifically confirmed	To test
TC-04	Attempt login with a nonexistent username.	Login fails and no timestamp is updated.	Not specifically confirmed	To test
TC-05	Log in successfully a second time.	Timestamp changes to the latest successful login.	Not specifically confirmed	To test
TC-06	Retrieve the student list from the API.	Response includes last_login for each student record.	Check API response	To confirm
TC-07	Open the Admin Dashboard.	Last Login column is visible and styled.	Reported working	Pass reported
TC-08	View an account with NULL last_login.	Dashboard displays “Never logged in”.	UI behavior implemented	Verify data case
TC-09	View an account with a timestamp.	Dashboard displays readable date and time.	UI behavior implemented	Verify data case
TC-10	Search and navigate the student list.	Existing list/search behavior remains operational.	Not specifically confirmed	To test
TC-11	Use existing Edit and Delete actions.	Existing CRUD operations continue to work.	Not specifically confirmed	To test
TC-12	Verify JWT login response.	Existing successful response and token are retained.	Code path retained	Regression check

3.3 Test Summary

The dashboard styling and Last Login display were reported as working. Not all authentication edge cases and regression cases were explicitly confirmed in the conversation, so they should not be marked as passed without executing them. Record the final pass/fail counts after running the test cases.

Metric	Result
Total test cases	12
Confirmed working from user feedback	Dashboard/CSS appearance; feature reported OK
Passed	Record after testing
Failed	Record after testing
Pending verification	Database evidence, failed-login behavior, API payload, regression cases

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	users table contains nullable last_login field.	Database schema / DESCRIBE users	To verify
AC-002	Successful student login updates last_login.	Login and database query	Reported feature working; capture evidence
AC-003	Failed login does not update last_login.	Compare timestamp before/after failed login	To test
AC-004	Student retrieval returns last_login.	Inspect API response	To verify
AC-005	Admin Dashboard shows Last Login column.	UI inspection	Reported working
AC-006	NULL timestamp displays “Never logged in”.	Test a NULL account	Verify in running UI
AC-007	Timestamp is readable and styled.	UI inspection	Reported CSS working
AC-008	Existing authentication/JWT flow remains operational.	Login regression test	To verify
AC-009	Existing student CRUD/search remains operational.	CRUD and search regression tests	To verify

5. Evidence to Attach

Figure 1 – Database column: Screenshot of DESCRIBE users showing last_login.

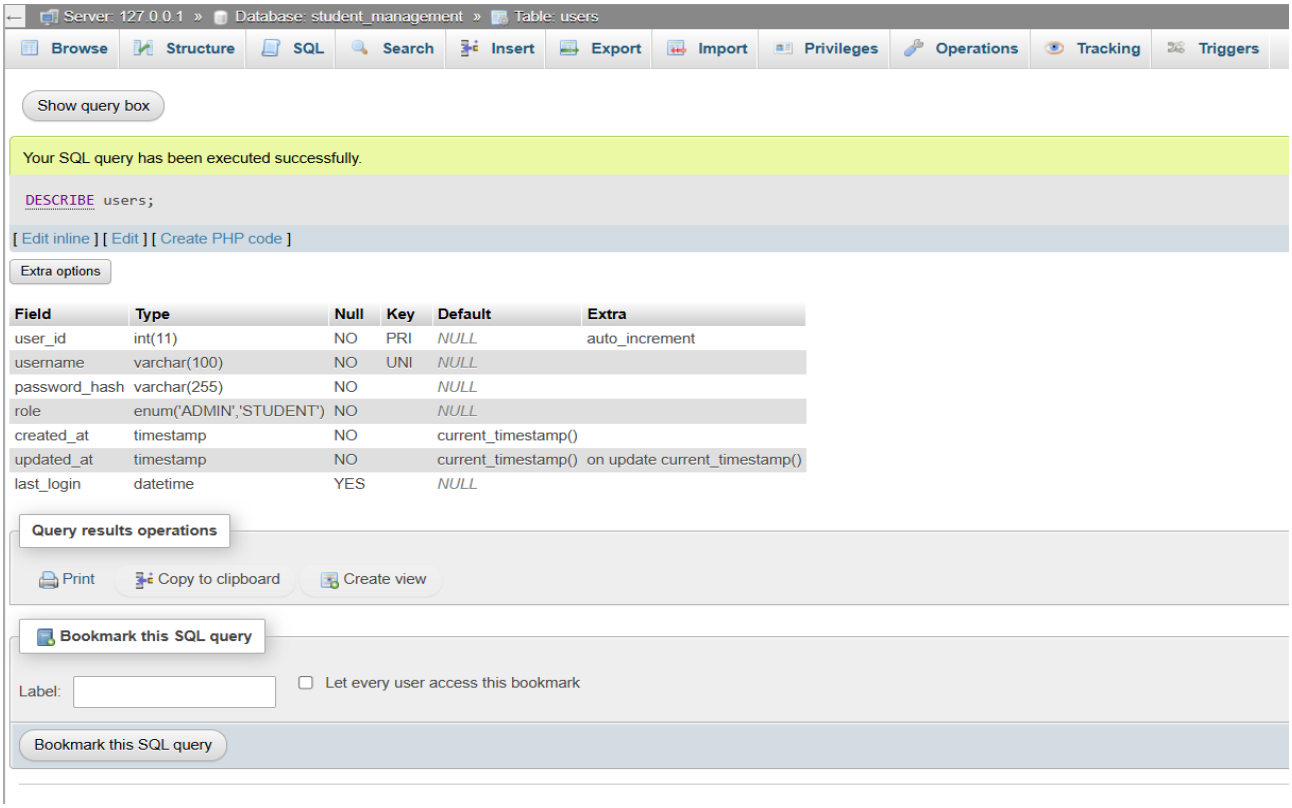


Figure 2 – Successful student login: Screenshot of successful login and a database query showing the updated timestamp.

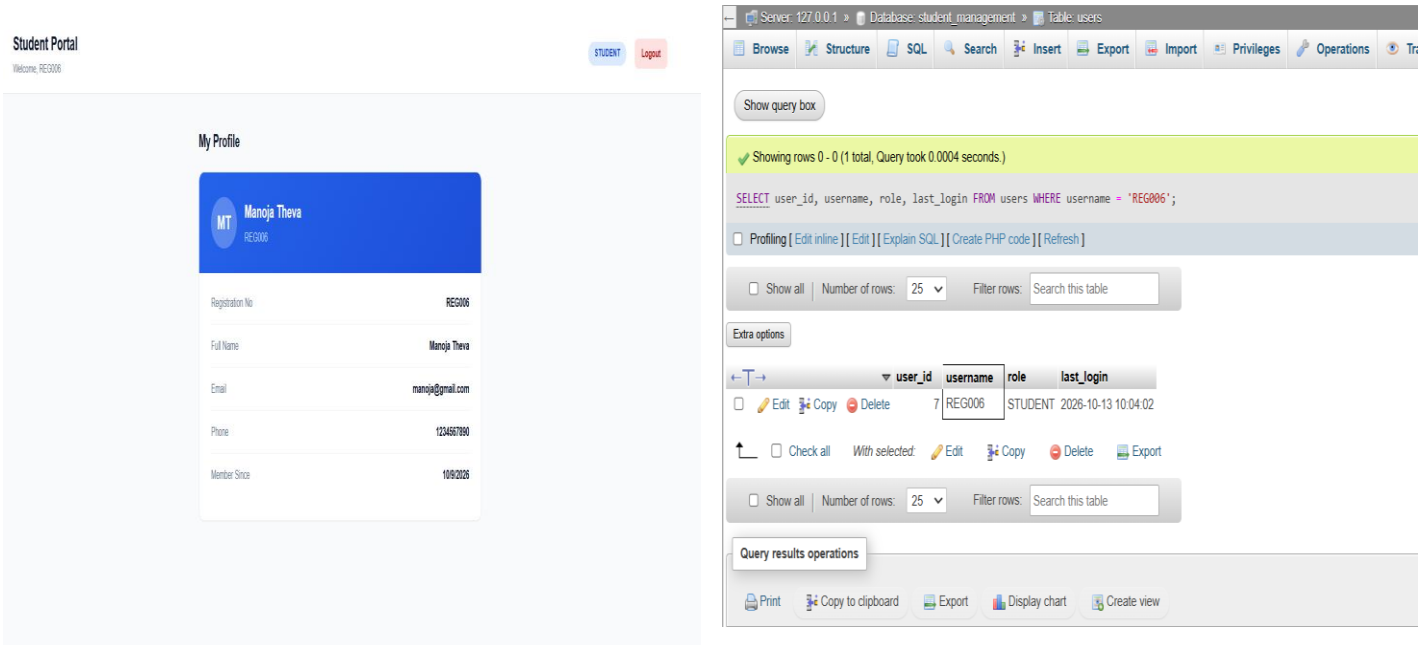


Figure 3 – Failed login: Screenshot of failed authentication plus evidence the timestamp did not change.

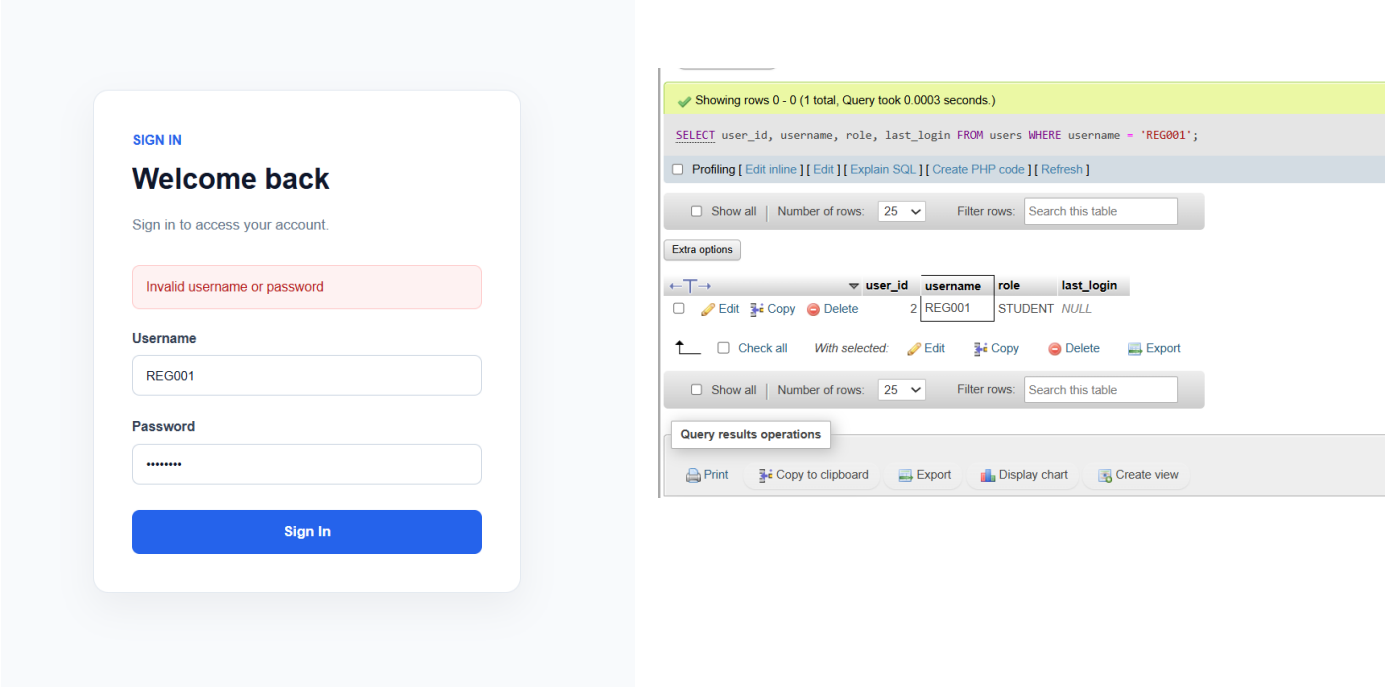


Figure 4 – Admin Dashboard: Screenshot showing the Last Login column and a formatted timestamp.

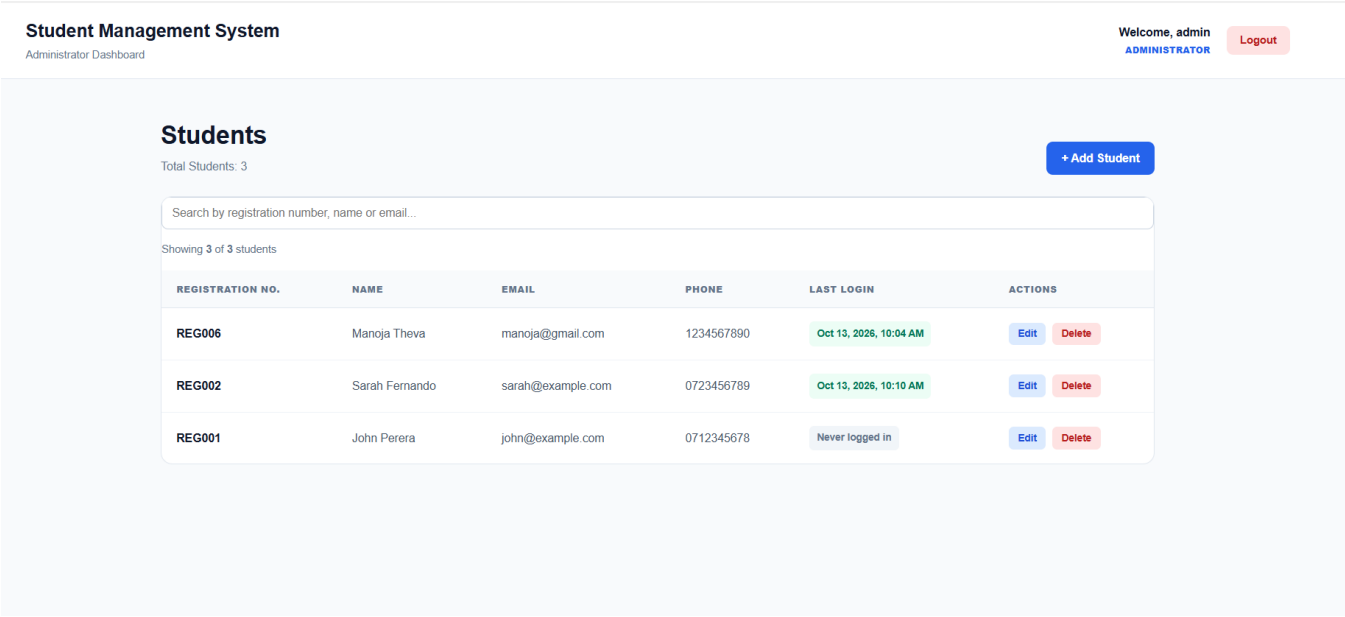
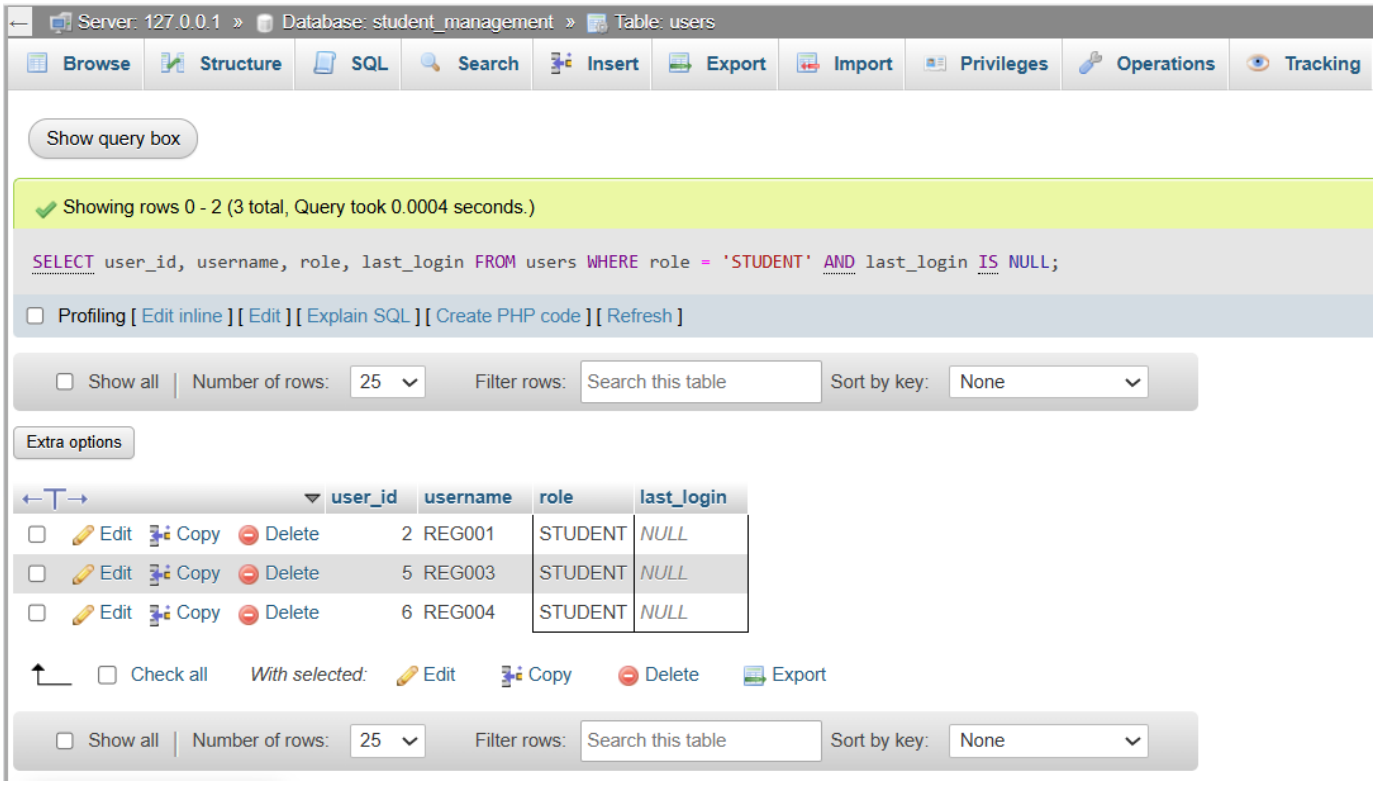


Figure 5 – Never logged in: Screenshot of a student whose last_login is NULL and whose row displays "Never logged in".



6. Final Result

The Last Login Tracking and Display feature has been added to the Student Management System code discussed. The user model updates the last_login timestamp after successful password verification, the student model retrieves the field with student details, and the Admin Dashboard displays a formatted timestamp or “Never logged in”. The dashboard CSS was also adjusted to style the new column and status labels. The user reported that the UI/CSS and feature were working. Formal acceptance should be finalized after the database evidence and remaining regression tests have been recorded.

6.1 Issues and Limitations

- The last_login column must exist in the database before the backend queries can use it.
- Accounts with no stored timestamp display “Never logged in” until a successful login is recorded.
- This change stores only the most recent login time, not a complete login history.
- NOW() uses the MySQL server's time setting; the browser formats the returned timestamp using its local date/time settings. Confirm timezone behavior if exact time matching is required.
- The supplied controller calls updateLastLogin for any user accepted by that login function. If only student accounts should be tracked, add a role check using the exact role value stored in the database.
- Add actual screenshots and update the test results before submitting the report.

Student Name: Y.Ugatharsini
Student ID: JF/ICT/24/33

Date: 13 October 2026
Signature: Y.Ugatharsini

— END OF REPORT —